

# **Behavioral Baselines and LLM-Assisted Explanation for Adaptive Fraud Detection:**

## ***A Research Proposal and System Design for a Distributed, Explainable Risk Intelligence Platform***

Vaibhav Kandpal

*B.Tech Computer Science, Dr. Akhilesh Das Gupta Institute of Professional Studies (GGSIPU), Delhi ·  
[github.com/Vaibhav20k](https://github.com/Vaibhav20k)*

### **Abstract**

Rule-based fraud detection systems degrade under adaptive adversaries: fraudsters observe which patterns trigger alerts and shift their behavior accordingly, producing a form of adversarial concept drift that static thresholds cannot track. This paper presents the motivation, threat model, mathematical formulation, and system architecture for a distributed, streaming fraud detection platform that (1) models per-user behavioral baselines rather than static rules, (2) treats anomaly scoring and drift adaptation as first-class, evaluable components, and (3) pairs every risk score with a natural-language explanation intended to support — rather than replace — human analyst judgment. We frame four research questions concerning false-positive reduction, analyst trust calibration, streaming latency, and adaptive-baseline performance relative to static thresholds. We report the current implementation status honestly: infrastructure, schema, and API contracts are complete; the inference, explanation, and evaluation components are designed but not yet built. We argue that the explainability and human-oversight questions this project raises — how to produce calibrated, faithful explanations for high-stakes automated decisions, and how to prevent explanation from becoming a substitute for genuine interpretability — are a small-scale, applied instance of problems central to AI safety research, and we discuss where this project's methodology does and does not transfer to frontier-model safety work.

### **1. Introduction and Motivation**

Financial fraud detection is usually framed as a classification problem: given a transaction, predict fraudulent or legitimate. This framing obscures three properties that make the problem harder, and more interesting, than standard supervised learning.

First, the data-generating process is non-stationary and partly adversarial. Fraudsters actively probe detection systems and adjust their behavior to evade them, producing what the concept-drift literature distinguishes from natural drift as adversarial or poisoning-driven drift — a malicious shift in the data distribution specifically intended to defeat a detector (Cardenas-Cartagena et al.; see also the taxonomy of adversarial concept drift types discussed by Korycki and Krawczyk). A detector trained once and left static will decay in precision as fraud patterns evolve.

Second, labels are delayed and incomplete. Confirmed fraud labels often arrive weeks after a transaction, via chargebacks or investigator review, which means naive online retraining is not

straightforward — a problem studied explicitly in the credit-card fraud literature under delayed supervised information.

Third, and most relevant to this proposal, false positives are not free. Every incorrectly flagged transaction consumes a human analyst's attention, and repeated false alarms produce alert fatigue, which degrades an analyst's ability to catch true positives later. This turns explanation quality into a functional requirement, not a nice-to-have: an analyst who receives a bare risk score with no rationale must either trust the system blindly or investigate from scratch, and neither is sustainable at production alert volumes.

This project's central hypothesis is that behavioral baselines, drift-aware scoring, and natural-language explanation are not three separate features but one coupled design problem: an explanation is only useful if it is grounded in the same features that drive the score, and a drift-adaptive score is only trustworthy to an analyst if its adaptation is itself explainable.

## 2. Related Work

Concept drift adaptation is a well-studied area; Gama et al. provide the standard survey, distinguishing drift by distributional type (covariate shift, prior shift, real concept drift) and temporal type (sudden, gradual, incremental, recurring). Work specific to fraud detection has shown that adversarial drift in the fraudulent class tends to be gradual and incremental, as fraudsters increase successful attack frequency while reducing detected attack frequency, which distinguishes it from the sudden drift caused by benign platform changes.

A separate line of work treats concept drift itself as an attack surface: Korycki and Krawczyk show that drift detectors can be fooled by adversarially injected data that simulates drift without a genuine underlying distribution change, motivating detectors that are themselves robust to poisoning. This is directly relevant to the present system, since a naive adaptive baseline that retrains on any detected "drift" is vulnerable to exactly this class of attack.

On the explainability side, hybrid frameworks combining SHAP/LIME-style feature attribution with human-in-the-loop review have been shown to materially improve recall and reduce adversarial attack success in production-style fraud systems, though such gains are typically reported on proprietary institutional data and are not easily reproduced. More recent work has begun exploring LLM-based explanation and joint fraud/drift detection in conversational settings, which is closer in spirit to this project's use of an LLM as an explanation-generation layer rather than a classifier.

This project sits at the intersection of these three threads — adversarially-aware drift detection, delayed-label learning, and LLM-assisted explanation — but does not claim novelty in any one of them individually. Its contribution, if the implementation is completed and evaluated, would be an integrated system study of how these three components interact under realistic streaming constraints, rather than a new algorithm in any single area.

## 3. Threat Model

A fraud detection system's threat model differs from a typical ML security threat model in one important way: the adversary's goal is not to cause a misclassification of a single input (as in adversarial examples for image classifiers) but to alter the system's learned notion of "normal" behavior over time. We consider the following adversary classes:

- Account takeover — an attacker gains control of a legitimate account and must decide how quickly to deviate from the account's established baseline before detection.
- Synthetic identity fraud — an attacker constructs a new identity with no prior baseline, exploiting the cold-start weakness of any behavioral-baseline approach.
- Mule accounts and money laundering networks — individually low-risk transactions that are only anomalous in aggregate, across accounts, which a per-user baseline will not catch without a graph-level view.
- Slow adversarial drift — an attacker who deliberately moves in small increments to remain within an adapting baseline's tolerance, directly exploiting the adaptivity that makes behavioral baselines useful in the first place.
- Poisoning of the drift detector itself — injecting transactions designed to look like organic behavioral change, causing the system to widen its notion of "normal" around actual fraud.

The last two are the most theoretically interesting and the most likely to be underestimated: a system built to be adaptive is, by construction, more exposed to an adversary who can influence what it adapts to. Any evaluation of this system must include a poisoning-robustness check, not only accuracy on a static holdout set.

## 4. System Architecture

The platform is designed as a streaming pipeline: a Go gRPC ingestion gateway accepts transaction events and publishes them to Kafka; a feature engineering stage computes behavioral features against a maintained per-user baseline; an ML anomaly detection stage scores the transaction; an LLM explanation stage converts the score and its contributing features into a natural-language rationale; and results are persisted to PostgreSQL and surfaced through a monitoring dashboard. Prometheus and Grafana provide operational telemetry (latency, throughput, Kafka consumer lag) separately from model-quality telemetry.

The pipeline, in order:

- Client → Go gRPC Ingestion Gateway
- Kafka Event Stream
- Feature Engineering (behavioral baseline lookup and update)
- ML Anomaly Detection
- LLM Explanation Generation
- PostgreSQL persistence
- Dashboard and monitoring (Prometheus / Grafana)

The design choice to separate scoring (statistical/ML) from explanation (LLM) is deliberate: the LLM is not the decision-maker, and does not see raw account data beyond the already-computed feature deltas passed to it. This is intended to reduce the risk of the explanation layer

hallucinating a rationale that does not correspond to the actual features that drove the score — a known failure mode when LLMs are used post-hoc to explain a separate model's decision rather than being grounded in that model's own feature attributions.

## 5. Mathematical Formulation

For a user  $u$ , we maintain a behavioral baseline as a feature-wise summary rather than a raw transaction history:

$$B_u = \{ \mu_a, \sigma_a, H, M, L, \tau \}$$

where  $\mu_a$  and  $\sigma_a$  are the running mean and standard deviation of transaction amount for user  $u$ ,  $H$  is a distribution over historical transaction hours,  $M$  is the set of previously seen merchants,  $L$  is the set of previously seen device/location fingerprints, and  $\tau$  is a drift-adaptation rate controlling how quickly the baseline updates in response to new, non-anomalous behavior.

Given a new transaction  $x$ , a risk function  $f$  maps the transaction and the baseline to a scalar risk score:

$$R = f(x, B_u)$$

and an explanation function  $g$  maps the transaction, score, and baseline to a natural-language rationale grounded in the specific features that contributed to  $R$ :

$$E = g(x, R, B_u)$$

The critical design constraint is that  $g$  must be a function of the same feature attributions that determine  $R$  — for example, the magnitude of  $(x\_amount - \mu_a)/\sigma_a$ , whether the merchant is in  $M$ , whether the device fingerprint is in  $L$  — rather than an independent LLM inference over the raw transaction. This is what distinguishes the explanation as faithful (grounded in the actual decision procedure) rather than merely plausible (a fluent but potentially unfaithful post-hoc narrative), a distinction the explainable-AI literature treats as critical and one this project has not yet empirically verified for its own pipeline.

Baseline adaptation itself is not treated as unconditional. A candidate update to  $B_u$  is only accepted if it passes a drift-plausibility check — intended to address the poisoning threat in Section 3 — rather than being applied to every non-anomalous transaction. The design of that check (a robust, ideally poisoning-aware drift detector rather than a naive exponential moving average) is one of the unresolved design questions this project has not yet settled empirically.

## 6. Research Questions

- RQ1 — Do behavioral, per-user baselines reduce false-positive rate relative to static, population-level thresholds, at matched recall?
- RQ2 — Do LLM-generated, feature-grounded explanations measurably change analyst response time or accuracy relative to a bare risk score, and are they faithful to the underlying feature attributions rather than merely plausible-sounding?

- RQ3 — Can the full streaming pipeline (ingestion → feature engineering → scoring → explanation) sustain sub-100ms end-to-end latency at realistic transaction throughput, or does the explanation stage become the bottleneck?
- RQ4 — Do adaptive baselines with a poisoning-aware drift check outperform both static thresholds and naive (unguarded) adaptive baselines under a simulated slow-drift adversary?

RQ4 is, in our view, the most research-relevant question, since it directly tests whether the adaptivity that makes behavioral baselines useful is exploitable by the adversary described in Section 3 — and whether the proposed poisoning-plausibility check actually mitigates that exploit or merely adds latency without improving robustness.

## 7. Implementation Status

In the interest of accurately representing the current state of this work rather than the intended end state, the table below summarizes what has been built versus what remains planned. No evaluation results exist yet, because the components required to produce them (inference engine, explanation engine) are not yet implemented.

| Component                                    | Status   | Notes                                                      |
|----------------------------------------------|----------|------------------------------------------------------------|
| Docker infrastructure                        | Complete | Multi-service local orchestration (Postgres, Redis, Kafka) |
| Database schema                              | Complete | transactions, user_baselines, anomaly_logs                 |
| gRPC API contract + generated protobuf       | Complete | Ingestion gateway interface defined                        |
| Go module, config, logging, server bootstrap | Complete | Core service scaffolding                                   |
| Transaction persistence                      | Planned  | Not yet implemented                                        |
| Kafka producer / streaming ingestion         | Planned  | Not yet implemented                                        |
| ML anomaly detection engine                  | Planned  | Not yet implemented                                        |
| LLM explanation engine                       | Planned  | Not yet implemented                                        |
| Evaluation on labeled data                   | Planned  | No results exist yet                                       |
| Dashboard / monitoring                       | Planned  | Not yet implemented                                        |

This is stated plainly because the research questions in Section 6 are only answerable once the corresponding components exist; at present this document constitutes a research design and system architecture, not an empirical results paper.

## 8. Evaluation Plan

Once implemented, the system will be evaluated along two axes: detection quality and system quality. Detection quality will be measured using ROC-AUC, precision, recall, and F1 on a held-out, time-ordered split (to avoid leaking future information into baseline construction, a common and easy-to-miss error in fraud-detection evaluation). System quality will be measured using end-to-end latency, throughput, Kafka consumer lag, and resource utilization under load.

For RQ4 specifically, evaluation will include a synthetic slow-drift adversary — an attacker agent that incrementally shifts transaction amount, timing, or merchant category within the bounds

the current baseline tolerates — to directly test whether the drift-plausibility check catches gradual adversarial adaptation that a naive adaptive baseline would silently absorb. For RQ2, evaluation will require either a small human study or, at minimum, a faithfulness check comparing the explanation's stated contributing features against the actual top feature attributions from the scoring model, since a fluent explanation that cites the wrong features would be worse than no explanation at all.

## 9. Relevance to AI Safety Research

This project is applied financial ML, not frontier-model alignment research, and we do not want to overstate the connection. That said, several of its design questions are small-scale, empirically tractable instances of problems that recur in AI safety research more broadly:

- Faithful vs. plausible explanation — the concern in Section 5 that an LLM explanation layer may produce a fluent rationale that does not correspond to the model's actual decision procedure is a miniature version of the interpretability question of whether a model's stated reasoning reflects its actual computation.
- Human oversight under alert fatigue — the motivation in Section 1, that excessive false positives degrade an analyst's ability to catch true positives, is a concrete instance of the general problem of maintaining meaningful human oversight of an automated system whose error rate is nonzero.
- Adversarial exploitation of adaptivity — RQ4 asks whether a system's capacity to update its own beliefs about "normal" behavior can itself be turned into an attack surface, which is structurally related to concerns about reward hacking or specification gaming in systems that are permitted to shift their own operating boundaries over time.
- Distribution shift and non-stationary deployment environments — the concept-drift framing throughout this paper is a specific case of the general safety concern that a system evaluated under one distribution may behave unpredictably once deployed into a shifting one.

We present these connections as motivation for why this project was worth the design effort, not as a claim that fraud detection and frontier AI safety are the same research area. The methodologies, stakes, and adversary sophistication differ substantially. We would characterize this work as evidence of research instincts — threat modeling before implementation, explicit evaluation criteria, honesty about what is and is not yet demonstrated — that we believe transfer, rather than as safety research in itself.

## 10. Limitations

- No results yet exist. Every claim in Sections 5–9 about what the system will do is a design intention, not a measured outcome, and should be read as such.
- No labeled fraud dataset has been selected or secured; public fraud datasets are scarce due to confidentiality constraints, and this remains an open practical problem for the evaluation plan in Section 8.
- The faithfulness of LLM-generated explanations to underlying feature attributions is asserted as a design goal in Section 5 but has no empirical verification method implemented yet.

- The poisoning-plausibility check for baseline adaptation (Section 5) is described only at the level of a design constraint; a specific, evaluated algorithm has not been chosen.
- This is single-author, non-institutional work without external review, replication, or a red-team evaluation of the threat model in Section 3.

## 11. Future Work

Immediate next steps, in priority order, are: (1) implement transaction persistence and the Kafka producer to complete the ingestion path; (2) implement a baseline anomaly detector (e.g., an Isolation Forest or a simple statistical z-score model) to produce the first real risk scores and unblock evaluation; (3) implement the LLM explanation layer with an explicit faithfulness check against feature attributions; (4) run the RQ4 poisoning/slow-drift experiment, since it is the most safety-relevant and currently the least implemented. Longer-term directions include graph-level detection for mule-account networks, online/continual learning under delayed labels, and a small human study of analyst response to explained versus unexplained alerts.

## References

- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4).
- Korycki, Ł., & Krawczyk, B. (2023). Adversarial concept drift detection under poisoning attacks for robust data stream mining. *Neural Computing and Applications / PMC*.
- Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2015). Credit card fraud detection and concept-drift adaptation with delayed supervised information.
- Machine Learning for Fraud Detection in E-Commerce: A Research Agenda. (2021). *arXiv:2107.01979*.
- Mai, T-D., Hoang, K., Baigutanova, A., Alina, G., & Kim, S. Customs Fraud Detection in the Presence of Concept Drift. *arXiv:2109.14155*.
- Adversarial Attacks for Drift Detection. (2024). *arXiv:2411.16591*.
- Domain Knowledge-Enhanced LLMs for Fraud and Concept Drift Detection. (2026). *MDPI Electronics*, 15(3), 534.
- Joint Detection of Fraud and Concept Drift in Online Conversations with LLM-Assisted Judgment. (2025). *arXiv:2505.07852*.